LAUREA MAGISTRALE
UNIVERSITÀ DEGLI STUDI DI MILANO-BICOCCA

Dipartimento di **Informatica, Sistemistica e Comunicazione**

Laurea magistrale in **informatica**

Titolo da decidere

Francesco Romeo
Matricola: 885880

Relatrice: **Dott.ssa Elisabetta Fersini**

Co-relatore: **Dottor Marco Loregian**

Anno Accademico **2025/2026**

Sommario

"abstract"

Indice

Introduzione **ii**

1 Stato dell'Arte e Background Tecnologico **1**
1.1 Architetture Transformer . . . 1
1.1.1 Dalle reti neurali ricorrenti ai Transformer . . . 1
1.1.2 Il meccanismo di Self-Attention . . . 1
1.1.3 Architettura encoder-decoder e varianti decoder-only . . . 2
1.1.4 Limiti in contesto zero-shot e motivazione del RAG . . . 2
1.2 LLM: panorama e analisi comparativa . . . 2
1.2.1 Modelli proprietari . . . 3
1.2.2 Motivazioni per l'adozione di modelli locali . . . 4
1.2.3 Modelli open-weight per uso on-premise . . . 4
1.2.4 Quantizzazione e inferenza on-premise . . . 5
1.2.5 Ollama: server di inferenza locale . . . 6
1.3 Retrieval-Augmented Generation (RAG) . . . 6
1.3.1 Motivazione e definizione . . . 6
1.3.2 Architettura del sistema RAG . . . 6
1.3.3 Modelli di embedding . . . 7
1.3.4 Database vettoriali . . . 8
1.3.5 Tecniche di adattamento al dominio . . . 8
1.4 Framework di orchestrazione: Haystack . . . 9
1.5 Sintesi e posizionamento del lavoro . . . 10

2 Analisi dei Requisiti e Progettazione del Sistema **11**

3 Implementazione del Sistema RAG **12**

4 Metodologia di Ricerca e Ottimizzazione **13**

5 Analisi Sperimentale e Valutazione dei Risultati **14**

Conclusioni e Sviluppi Futuri **15**

A Annotazioni e Convenzioni **16**

B Codice Utilizzato **17**

Bibliografia **18**

Ringraziamenti **20**

Introduzione

* Contesto: Diffusione dei Large Language Models (LLM) e le sfide dell'adozione in ambito corporate (privacy, allucinazioni, aggiornamento dei dati). —¿ giustificare le affermazioni con citazioni e dati. * Il Problema: Gestione della conoscenza non strutturata e valorizzazione del materiale formativo (Gilde) in Intré. —¿ descrizione dell'azienda e dello use-case. * Obiettivo della tesi: Progettazione e sviluppo di un sistema RAG (Retrieval-Augmented Generation) con focus sull'ottimizzazione dell'inferenza e dell'accuratezza nel dominio specifico. * Estrema sintesi dei risultati ottenuti * NOTA: essere esplicativi sotto tutti i punti di vista, deve essere una versione sintetica della testi - un riassunto - con l'aggiunta delle motivazioni del progetto.

Capitolo 1

Stato dell'Arte e Background Tecnologico

Il presente capitolo fornisce le fondamenta teoriche e tecnologiche necessarie alla comprensione del sistema progettato nei capitoli successivi. Vengono discusse le architetture Transformer, su cui si basano tutti i modelli linguistici (*Large Language Models*, LLM) considerati nel progetto, analizzando sia i modelli aperti sia quelli proprietari, con particolare attenzione alle soluzioni adatte all'esecuzione on-premise; viene poi illustrato il paradigma *Retrieval-Augmented Generation* (RAG) nella sua architettura complessiva e nelle tecniche di adattamento al dominio, come risposta strutturata ai limiti intrinseci dei LLM.

1.1 Architetture Transformer

1.1.1 Dalle reti neurali ricorrenti ai Transformer

Il modello Transformer, introdotto da Vaswani et al. nel 2017 [20], ha ridefinito lo stato dell'arte nel trattamento del linguaggio naturale (*Natural Language Processing*, NLP). I sistemi precedenti erano basati su reti neurali ricorrenti (*Recurrent Neural Networks*, RNN) e, più specificamente, su architetture LSTM (*Long Short-Term Memory*) [8], che elaboravano i dati in modo sequenziale, accumulando uno stato nascosto ad ogni passo temporale. Tale formulazione introduceva due limitazioni strutturali: l'impossibilità di parallelizzare il calcolo durante l'addestramento e la difficoltà di mantenere dipendenze a lungo termine in sequenze di lunghezza elevata, problema noto come *vanishing gradient*.

Il Transformer risolve entrambi i problemi abbandonando la ricorrenza in favore di un meccanismo di *self-attention* globale, che consente a ciascun elemento di una sequenza di relazionarsi direttamente con tutti gli altri, indipendentemente dalla distanza posizionale.

1.1.2 Il meccanismo di Self-Attention

Il nucleo computazionale del Transformer è il meccanismo di *attention* scalato. Data una sequenza di token rappresentati come vettori di dimensione d_{model}, vengono calcolate tre proiezioni lineari

per ogni elemento: la query $\mathbf{Q}$, la chiave $\mathbf{K}$ e il valore $\mathbf{V}$. Il punteggio di attenzione è definito come [20]:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{1.1}$$

dove d_k è la dimensione dei vettori di chiave e il fattore $1/\sqrt{d_k}$ previene la saturazione della funzione softmax in presenza di prodotti scalari di grande magnitudine.

Il meccanismo viene replicato su più *teste* in parallelo, ciascuna delle quali apprende una proiezione diversa dello stesso spazio di input: si parla di *Multi-Head Attention* (MHA). La concatenazione degli output di h teste permette al modello di catturare simultaneamente relazioni sintattiche, semantiche e posizionali:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\,\mathbf{W}^O \tag{1.2}$$

con $\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$.

La posizione dei token nella sequenza non è catturata dall'attention; viene iniettata tramite *positional encoding*: vettori sommati agli embedding di input che forniscono al modello informazioni sull'ordine relativo degli elementi.

1.1.3 Architettura encoder-decoder e varianti decoder-only

L'architettura originale del Transformer è di tipo encoder-decoder: un encoder trasforma la sequenza di input in una rappresentazione contestuale densa, mentre un decoder la utilizza — attraverso un meccanismo di *cross-attention* — per generare la sequenza di output, ed è particolarmente adatta a task *sequence-to-sequence* come la traduzione automatica [20].

I modelli linguistici generativi contemporanei adottano prevalentemente un'architettura *decoder-only*, nella quale l'attenzione è applicata in modo *causale* (o *masked*): ogni token dipende esclusivamente dai token che lo precedono nella sequenza. Questo vincolo è necessario per l'addestramento autoregressivo, dove il modello apprende a predire il token successivo dato il contesto precedente.

1.1.4 Limiti in contesto zero-shot e motivazione del RAG

Nonostante l'efficacia dei modelli Transformer pre-addestrati, la loro applicazione in contesti aziendali specifici presenta limitazioni strutturali non trascurabili, sintetizzate nella Tabella 1.1.

Questi tre fattori motivano l'adozione del paradigma RAG, discusso nella Sezione 1.3, come alternativa pragmatica al fine-tuning per l'adattamento al dominio.

1.2 LLM: panorama e analisi comparativa

L'ecosistema degli LLM si è diversificato rapidamente a partire dal 2022, con una netta polarizzazione tra modelli proprietari, accessibili esclusivamente tramite API, e modelli open-weight, i cui pesi sono pubblicamente scaricabili e utilizzabili on-premise — talvolta con licenze che ne limitano l'uso commerciale. La Tabella 1.2 fornisce una visione d'insieme dei principali modelli considerati.

Tabella 1.1: Limiti strutturali degli LLM in contesti aziendali e relative strategie di mitigazione.

Limitazione	**Descrizione**	**Strategia di mitigazione**
Conoscenza statica	I parametri codificano la conoscenza fino al *knowledge cutoff*; nessun accesso a dati interni o aggiornati dopo l'addestramento.	RAG: recupero dinamico da knowledge base esterna, aggiornabile senza riaddestrare il modello.
Allucinazioni [10]	Il modello genera testo plausibile ma fattualmente errato su informazioni assenti dalla distribuzione di addestramento.	RAG con *grounding*: la risposta è ancorata a documenti recuperati e verificabili.
Privacy dei dati	Il fine-tuning su modelli cloud espone dati riservati a infrastrutture di terze parti, con rischi di conformità normativa.	Esecuzione on-premise con modelli open-weight; nessuna trasmissione di dati verso servizi esterni.

Tabella 1.2: Panorama comparativo dei principali LLM (2024–2025). † Esecuzione garantita on-premise senza dipendenze cloud; ‡ pesi scaricabili pubblicamente; MoE = *Mixture of Experts*.

Modello	**Sviluppatore**	**Licenza**	**Accesso**	**Architettura / Param. attivi**	**Ctx**	**Multi-modale**
GPT-5 / o3	OpenAI	Proprietaria	API cloud	Dense, n.d.	128k	✓
Claude 4 Sonnet	Anthropic	Proprietaria	API cloud	Dense, n.d.	200k	✓
Gemini 3 Pro	Google DM	Proprietaria	API cloud	Dense, n.d.	1M	✓
Llama 4 Scout	Meta AI	Open-weight‡	Self-hosted†	MoE, 17B	10M	✓
Mistral Large 3	Mistral AI	Apache 2.0‡	Self-hosted†	MoE, ~40B	128k	✓
Qwen3-8B	Alibaba Cloud	Apache 2.0‡	Self-hosted†	Dense, 8B	128k	–

1.2.1 Modelli proprietari

I modelli proprietari rappresentano lo stato dell'arte sulle prestazioni su benchmark generali, ma presentano vincoli rilevanti per l'adozione aziendale in contesti con requisiti di riservatezza.

GPT-5 e OpenAI o-series. La famiglia GPT-5 [16] rappresenta l'attuale frontiera per la pianificazione complessa e l'integrazione multimodale avanzata. La serie "o" (o1, o3) integra nativamente il *chain-of-thought* durante l'inferenza, migliorando le prestazioni in ambito scientifico e logico-matematico [29]. L'erogazione è vincolata ad API cloud proprietarie; in contesti enterprise è possibile operare tramite *Azure OpenAI Service*, che garantisce isolamento logico dei dati e assenza di utilizzo per il riaddestramento dei modelli base.

Claude 4 (Anthropic). I modelli Claude si distinguono per l'approccio alla *Constitutional AI* [1], un protocollo di addestramento guidato da principi etici per garantire l'allineamento del modello. Come per GPT-5, l'architettura è nativamente cloud-only; Anthropic offre garanzie contrattuali

enterprise tramite la modalità *Zero Data Retention* e l'integrazione con Amazon Bedrock e Google Cloud Vertex AI [22].

Gemini (Google DeepMind). La famiglia Gemini [24] si distingue per la multimodalità nativa e per context window fino a 1M di token (Gemini 3 Pro), con capacità agentiche avanzate. Come gli altri modelli proprietari, è disponibile esclusivamente in modalità cloud-hosted.

1.2.2 Motivazioni per l'adozione di modelli locali

Nonostante le prestazioni dei modelli proprietari analizzati, il sistema descritto in questa tesi adotta modelli eseguiti interamente in locale (*self-hosted*). La decisione poggia su tre motivazioni strutturali, sintetizzate nella Tabella 1.3.

Tabella 1.3: Motivazioni per l'adozione di modelli LLM on-premise.

Motivazione	**Descrizione**	**Riferimento normativo / letteratura**
Sovranità del dato e conformità	L'esecuzione locale elimina la trasmissione dei dati verso infrastrutture di terze parti, anche in presenza di garanzie contrattuali (ZDR, DPA).	GDPR [6]; rischi del cloud nella trattazione di dati sensibili.
Costi operativi nel lungo periodo	Il modello di pricing a consumo (per token) dei provider cloud può risultare più oneroso rispetto all'ammortamento dell'infrastruttura hardware locale in scenari ad alto volume.	[2]; rischio di *vendor lock-in*.
Controllo e personalizzazione	I modelli open-weight (LLaMA, Mistral, Qwen) consentono fine-tuning supervisionato e allineamento su dati propri, senza restrizioni imposte dalle policy d'uso dei sistemi proprietari.	[19, 11, 17]

Per queste ragioni, la trattazione si concentra nelle sezioni successive sull'analisi di modelli eseguibili localmente, riservando ai sistemi proprietari il ruolo di riferimento comparativo per la valutazione delle prestazioni nel Capitolo **??**.

1.2.3 Modelli open-weight per uso on-premise

Llama 4 (Meta AI). Rilasciata nell'aprile 2025, la famiglia Llama 4 [26] segna il passaggio definitivo di Meta all'architettura nativa *Mixture of Experts* (MoE) e alla multimodalità integrata. Le varianti *Scout* e *Maverick* bilanciano parametri attivi (17B) e densità di esperti per gestire contesti fino a 10 milioni di token. La licenza open-weight adottata da Meta presenta clausole restrittive per i grandi operatori e limitazioni per i soggetti residenti nell'Unione Europea, un fattore critico da valutare in fase di adozione.

Mistral 3 (Mistral AI). Mistral AI ha consolidato nel 2025 un'offerta orientata alla massima flessibilità d'integrazione [27]. Il modello di punta, *Mistral Large 3*, è un modello MoE multilingue affiancato dalla serie *Ministral* (3B–14B), ottimizzata per l'esecuzione su hardware locale. La distribuzione sotto licenza Apache 2.0 rende l'intera famiglia la soluzione di riferimento per lo sviluppo di sistemi aperti e personalizzabili.

Qwen3 (Alibaba Cloud). La generazione Qwen3 [31] introduce un approccio ibrido al ragionamento, alternando modalità *thinking* per task logici e modalità diretta per l'efficienza. La famiglia spazia da modelli ultra-compatti (0.6B) a varianti MoE con finestre di contesto estese. Come approfondito nel Capitolo **??**, la variante da 8B è stata selezionata per il sistema qui presentato per il favorevole rapporto tra qualità semantica e velocità di inferenza su hardware CPU-only.

1.2.4 Quantizzazione e inferenza on-premise

Un aspetto centrale per l'esecuzione locale di LLM è la **quantizzazione**: la riduzione della precisione numerica dei pesi del modello da FP32 o FP16/BF16 a rappresentazioni a bassa precisione (8-bit, 4-bit). La Tabella 1.4 riassume l'impatto delle principali tecniche su memoria e qualità per un modello da 7B parametri.

Tabella 1.4: Impatto della quantizzazione su occupazione di memoria e qualità per un modello da 7B parametri. I valori sono indicativi e tratti da [5, 7].

Formato	**Bit/peso**	**VRAM (7B)**	**Degr. qualità**	**Tecnica**
FP32	32	~28 GB	baseline (0%)	Standard
FP16/BF16	16	~14 GB	≈ 0%	Standard
GPTQ	8	~8 GB	<1%	[7]
GGUF Q4_K_M	4	~4 GB	1–3%	llama.cpp
NF4 (QLoRA)	4	~4 GB	<1%	[5]

L'impatto della quantizzazione sulla qualità della generazione in contesti domain-specific viene analizzato nel Capitolo **??**.

> In data 24 marzo 2026, Google Research ha presentato TurboQuant[25], un framework di compressione data-oblivious progettato per abbattere i colli di bottiglia mnemonici della KV Cache[1] nei Large Language Models (LLM). La tecnologia si distingue per la capacità di ridurre il footprint della memoria di un fattore di almeno 6x, garantendo al contempo uno speedup computazionale fino a 8x su acceleratori hardware di ultima generazione, senza degradazione apprezzabile della precisione.
>
> Il core metodologico di TurboQuant si articola in due stadi sequenziali: PolarQuant [2], che normalizza la distribuzione dei dati tramite una proiezione in coordinate polari, e

[1] KV Cache: Struttura dati utilizzata per memorizzare i tensori Key e Value durante l'inferenza autoregressiva degli LLM. Poiché la sua dimensione cresce linearmente con la lunghezza del contesto ($O(n)$), essa rappresenta il principale limite fisico per il deployment di modelli a lungo contesto su hardware con VRAM limitata.

[2] PolarQuant: Tecnica di quantizzazione vettoriale che trasforma i dati dallo spazio cartesiano a quello polare. Applicando una rotazione casuale, i vettori vengono mappati su una distribuzione Beta altamente prevedibile, eliminando l'overhead di memorizzare costanti di quantizzazione (scale e zero-point) per ogni blocco di dati.

il Quantized Johnson-Lindenstrauss (QJL)[3], un correttore di errore a 1-bit che agisce come stimatore non distorto per i prodotti scalari. Data la sua natura training-free e l'efficienza nel preservare la fedeltà dei logit di attenzione, l'integrazione di TurboQuant apre prospettive inedite per l'estensione delle finestre di contesto in locale, risultando un pilastro tecnologico per l'evoluzione del sistema proposto in questo lavoro.

1.2.5 Ollama: server di inferenza locale

Per il progetto in esame, l'infrastruttura di inferenza è gestita tramite **Ollama** [28], un server open source che espone modelli locali in formato GGUF tramite un'API REST compatibile con la specifica *OpenAI Chat Completions*. Ollama semplifica il ciclo di vita dei modelli locali: l'installazione avviene tramite singolo comando (`ollama pull <modello>`), la gestione del contesto e del formato di prompt è automatizzata, e il server è distribuibile come container Docker. Nel contesto di questo progetto, Ollama serve sia il modello di chat (`qwen3:8b`) sia il modello di embedding (`nomic-embed-text`), il cui ruolo è descritto nella Sezione 1.3.3.

1.3 Retrieval-Augmented Generation (RAG)

1.3.1 Motivazione e definizione

Il paradigma *Retrieval-Augmented Generation* (RAG), formalizzato da Lewis et al. [12], nasce per superare il limite fondamentale dei LLM come sistemi a conoscenza chiusa (*closed-book*): la conoscenza del modello è codificata nei parametri e non può essere aggiornata senza un nuovo ciclo di addestramento o fine-tuning.

In un sistema RAG, il processo di generazione è arricchito da una fase di *retrieval* dinamico: prima di invocare l'LLM viene eseguita una ricerca in una base di conoscenza esterna per recuperare i documenti pertinenti alla query. Tali documenti vengono iniettati nel contesto del prompt (*prompt augmentation*), permettendo al modello di fondare la propria risposta su informazioni aggiornate e verificabili, riducendo il rischio di allucinazioni [18].

1.3.2 Architettura del sistema RAG

Un sistema RAG si compone di due macro-pipeline distinte, messe a confronto nella Tabella 1.5 e illustrate nella Figura 1.1: la pipeline di **ingestione** (*ingestion pipeline*), eseguita offline, e la pipeline di **inferenza** (*query pipeline*), eseguita in tempo reale per ogni richiesta.

Pipeline di ingestione

La pipeline di ingestione trasforma i documenti grezzi in vettori persistiti nel database vettoriale. Le fasi principali sono le seguenti.

Pre-processing e chunking. Il documento viene suddiviso in unità testuali (*chunk*) di dimensione controllata. La strategia di chunking impatta direttamente sulla qualità del retrieval:

[3] QJL (Quantized Johnson-Lindenstrauss): Un'applicazione del lemma di Johnson-Lindenstrauss che permette di proiettare vettori in uno spazio a bassa dimensione preservando quasi isometricamente le distanze L^2. In TurboQuant, funge da termine di correzione per minimizzare l'errore sistematico introdotto dalla quantizzazione aggressiva (fino a 3.5 bit per canale).

Tabella 1.5: Le due pipeline del sistema RAG a confronto.

Fase	**Ingestion pipeline (offline)**	**Query pipeline (real-time)**
1	Pre-processing e chunking dei documenti sorgente	Embedding della query utente nello stesso spazio vettoriale dei documenti
2	Generazione degli embedding tramite modello locale (`nomic-embed-text`)	Ricerca ANN nel database vettoriale (*top-k*) con filtri su metadati
3	Indicizzazione in Qdrant con payload di metadati strutturati	Augmented generation: contesto recuperato + query → LLM
Trigger	Esecuzione manuale o schedulata (es. giornaliera)	Ad ogni richiesta utente
Latenza	Minuti / ore (elaborazione batch)	Millisecondi (inferenza real-time)

chunk troppo piccoli perdono il contesto locale, chunk troppo grandi diluiscono il segnale semantico e rischiano di superare la finestra di contesto del modello di embedding [4].

Generazione degli embedding. Ogni chunk viene trasformato in un vettore denso da un modello di embedding $f{:}\mathcal{T} \to \mathbb{R}^d$, tale che documenti semanticamente simili producano vettori vicini secondo la similarità coseno:

$$\mathrm{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|} \tag{1.3}$$

Indicizzazione nel database vettoriale. I vettori vengono persistiti in **Qdrant** [30], che implementa l'algoritmo HNSW (*Hierarchical Navigable Small World*) [13] per la ricerca approssimata (*Approximate Nearest Neighbor*, ANN). La scelta di Qdrant rispetto ad alternative quali pgvector, Chroma e Milvus è motivata nel Capitolo **??**.

Pipeline di inferenza

Embedding della query. La query dell'utente viene proiettata nel medesimo spazio vettoriale dei documenti indicizzati con lo stesso modello di embedding.

Retrieval. Viene eseguita una ricerca ANN in Qdrant per recuperare i k documenti più simili alla query. La ricerca può essere ristretta tramite *metadata filtering* a sottoinsiemi strutturalmente coerenti (ad esempio, documenti appartenenti a una specifica epoca temporale nel sistema i3Guild).

Augmented generation. Il contesto recuperato viene concatenato alla query nel prompt inviato all'LLM. Il modello genera la risposta basandosi sia sulla propria conoscenza parametrica sia sulle informazioni esterne fornite.

1.3.3 Modelli di embedding

La scelta del modello di embedding è una decisione architetturale con impatto diretto sulla qualità del retrieval. Il vincolo di esecuzione on-premise restringe la scelta a modelli distribuibili localmente tramite Ollama. La valutazione comparativa è stata condotta su quattro candidati — `nomic-embed-text`, `mxbai-embed-large`, `all-minilm` e `multilingual-e5-large` — secondo criteri di qualità in lingua italiana, occupazione di memoria, compatibilità con Ollama e punteggi sul

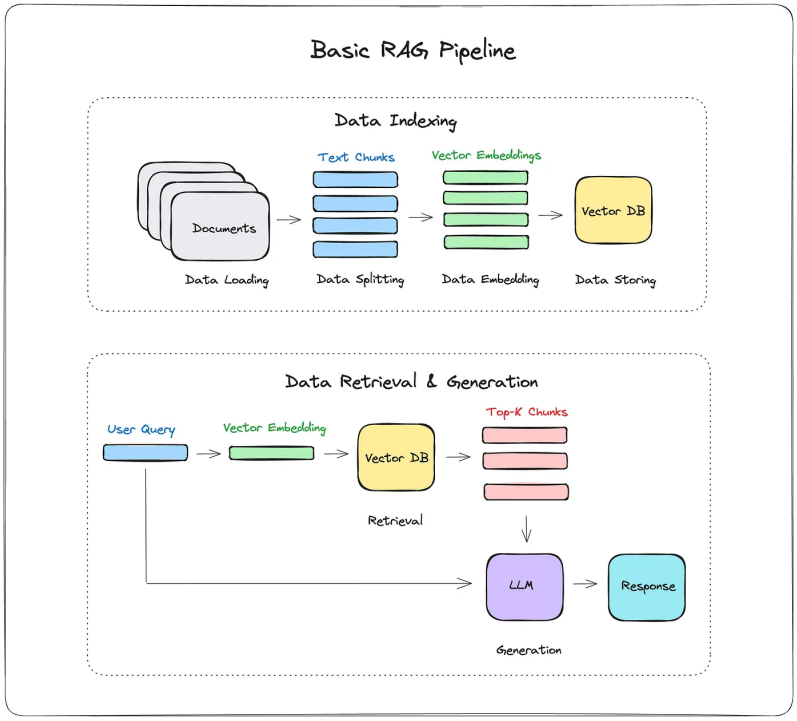

Figura 1.1: Schema della pipeline RAG: la sezione superiore rappresenta la fase di ingestione offline; la sezione inferiore la pipeline di inferenza in tempo reale [12].

benchmark MTEB (*Massive Text Embedding Benchmark*) [14]. L'analisi completa e la motivazione della scelta finale di `nomic-embed-text` [15] sono presentate nel Capitolo **??**.

1.3.4 Database vettoriali

Un database vettoriale è un sistema di storage specializzato per vettori densi ad alta dimensionalità, progettato per rispondere efficientemente a query di tipo *k-nearest neighbor* (kNN). A differenza dei database relazionali, dove le query sono esatte, la ricerca vettoriale è per natura approssimata: si cercano i vettori più vicini accettando un trade-off tra precisione (*recall*) e velocità (*throughput*).

L'algoritmo HNSW costruisce un grafo gerarchico multi-livello sullo spazio vettoriale, con complessità di ricerca $O(\log n)$ rispetto al numero di punti n — contro $O(n)$ della ricerca lineare esatta [13] — garantendo scalabilità anche in presenza di corpora di grandi dimensioni.

1.3.5 Tecniche di adattamento al dominio

L'adattamento di un LLM pre-addestrato a un dominio specifico può avvenire tramite approccio parametrico (*fine-tuning*) o non parametrico (RAG, *prompt engineering*). Questa sezione analizza le tecniche di fine-tuning con parametri ridotti, che costituiscono il riferimento teorico per il confronto discusso nel Capitolo **??**.

Fine-tuning con adattatori: LoRA

Il fine-tuning completo di un LLM richiede risorse computazionali proibitive. Hu et al. [9] hanno proposto *Low-Rank Adaptation* (LoRA), che vincola gli aggiornamenti dei pesi a matrici di rango basso. Dato un layer lineare con matrice $\mathbf{W}_0 \in \mathbb{R}^{m \times n}$, LoRA introduce:

$$\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \mathbf{BA} \tag{1.4}$$

con $\mathbf{A} \in \mathbb{R}^{r \times n}$, $\mathbf{B} \in \mathbb{R}^{m \times r}$ e $r \ll \min(m, n)$. I pesi originali $\mathbf{W}_0$ rimangono congelati; solo $\mathbf{A}$ e $\mathbf{B}$ vengono ottimizzati. Con $r = 8$ e $m = n = 4096$, il numero di parametri addestrabili si riduce di circa due ordini di grandezza rispetto al fine-tuning completo.

QLoRA: fine-tuning a 4-bit

Quantized LoRA (QLoRA) [5] estende LoRA combinando la quantizzazione a 4-bit del modello base con gli adattatori LoRA a precisione piena. I pesi vengono quantizzati in formato NF4 (*NormalFloat4*), ottimizzato per distribuzioni gaussiane tipiche dei pesi neurali; durante il forward pass vengono de-quantizzati dinamicamente in BF16, mentre gli adattatori operano sempre in BF16.

La Tabella 1.6 riassume i principali trade-off tra le due varianti.

Tabella 1.6: Confronto tra LoRA e QLoRA per un modello base da 7B parametri.

Caratteristica	**LoRA**	**QLoRA**
Precisione pesi base	FP16/BF16	NF4 (4-bit)
Precisione adattatori	FP16/BF16	BF16
Memoria GPU (modello 7B)	~14 GB	~4 GB
Overhead de-quantizzazione	assente	presente
Degradazione qualitativa stimata	trascurabile	<1% su benchmark principali

Prompt engineering e inference-time intervention

In alternativa al fine-tuning parametrico, è possibile adattare il comportamento di un LLM intervenendo sulla formulazione dell'input senza modificare i pesi. Tecniche come *few-shot prompting* [3] e *chain-of-thought prompting* [21] guidano il modello verso risposte più strutturate arricchendo il prompt con esempi o istruzioni di ragionamento intermedie. Nel sistema descritto in questa tesi, il prompt engineering è impiegato nella definizione del *system prompt* e nell'iniezione strutturata del contesto RAG; le strategie adottate e il loro impatto sulla qualità delle risposte sono analizzati nel Capitolo **??**.

1.4 Framework di orchestrazione: Haystack

La costruzione di sistemi RAG richiede l'integrazione di componenti eterogenei — modelli di embedding, database vettoriali, LLM, pipeline di pre-processing — attraverso interfacce coerenti e componibili. **Haystack** [23], sviluppato da deepset, è un framework Python open source progettato per questo scopo.

Nella versione 2.x adottata in questo progetto, ogni pipeline è un grafo diretto aciclico (DAG) di componenti con interfacce tipizzate, collegabili in modo dichiarativo. Sono disponibili integrazioni native con Qdrant (`QdrantDocumentStore`), Ollama (`OllamaDocumentEmbedder`, `OllamaTextEmbedder`, `OllamaGenerator`) e le principali operazioni di pre-processing (`DocumentCleaner`, `DocumentSplitter`). Il dettaglio implementativo della pipeline di ingestione e di retrieval è descritto nel Capitolo **??**.

1.5 Sintesi e posizionamento del lavoro

Il presente capitolo ha fornito il quadro teorico su cui si fonda il sistema descritto nei capitoli successivi. Le architetture Transformer costituiscono il substrato computazionale comune a tutti i componenti di generazione e di embedding; i loro limiti in contesti a conoscenza chiusa — conoscenza statica, allucinazioni, privacy — motivano l'adozione del paradigma RAG. La scelta di operare con modelli e infrastrutture interamente on-premise (Ollama, Qdrant, Haystack) risponde ai vincoli di sovranità del dato e sostenibilità computazionale identificati nell'analisi dei requisiti. Le tecniche di fine-tuning a parametri ridotti (LoRA, QLoRA) costituiscono il termine di confronto teorico per la valutazione delle strategie di adattamento al dominio nel Capitolo **??**.

Capitolo 2

Analisi dei Requisiti e Progettazione del Sistema

* Il caso d'uso Intré: Analisi dei dati aziendali (documenti tecnici, glossari, archivi Gilde). * Requisiti di Sistema: compatibilità con infrastrutture aziendali, privacy dei dati e costi computazionali. * Architettura della Soluzione: * Scelta dello stack tecnologico (es. LangChain/LlamaIndex, Ollama, Vector DB come Chroma o Pinecone). * Infrastruttura: Integrazione nel flusso i3Guild. * Nota: quando possibile usare schemi/tabelle/diagrammi.

Capitolo 3

Implementazione del Sistema RAG

* Pipeline di Ingestion: Pre-processing dei documenti, chunking strategies e generazione degli embedding. * Integrazione LLM: Configurazione dell'infrastruttura locale (on-premise) e gestione dei System Prompt. * Sviluppo dell'Agente Cognitivo: Logica di gestione del contesto e conversazione multi-turno. * Gestione fonti eterogenee: Integrazione di dati strutturati e non strutturati (esiti delle Gilde).

Capitolo 4

Metodologia di Ricerca e Ottimizzazione

* Ottimizzazione dell'Attention: Strategie per il trattamento del glossario tecnico aziendale. * Inference-Time Intervention: Sperimentazione sulla manipolazione dei vettori di attivazione (come discusso con la Professoressa). * Strategie di Quantizzazione: Impatto della quantizzazione (4-bit vs 8-bit) sulla qualità della generazione in contesti specialistici. * Fine-tuning (Opzionale): Analisi di fattibilità con risorse limitate (PEFT/

Capitolo 5

Analisi Sperimentale e Valutazione dei Risultati

* Dataset di Validazione: Definizione di una "Ground Truth" per le domande sulle Gilde. * Metriche Quantitative: * Retrieval: Hit Rate, MRR (Mean Reciprocal Rank). * Generation: Faithfulness, Answer Relevance (tramite framework come RAGAS). * Analisi Qualitativa: Studio dei casi di fallimento (allucinazioni, errori di retrieval) e feedback degli stakeholder di Intré. * Benchmarking: Confronto tra LLM diversi (es. OpenAI vs Llama locale).

Conclusioni e Sviluppi Futuri

* Sintesi del lavoro: Efficacia del sistema RAG nell'ecosistema aziendale. * Contributi originali: Risultati dell'analisi metodologica sull'attenzione e l'inferenza. * Limiti e lavori futuri: Agenti multi-modali, miglioramento continuo della base di conoscenza.

Appendice A

Annotazioni e Convenzioni

Text

Appendice B

Codice Utilizzato

Text

Bibliografia

Riferimenti Bibliografici

[1] Yuntao Bai et al. "Constitutional AI: Harmlessness from AI Feedback". In: *arXiv preprint arXiv:2212.08073* (2022). URL: https://arxiv.org/abs/2212.08073.

[2] Rishi Bommasani et al. "On the Opportunities and Risks of Foundation Models". In: *arXiv preprint arXiv:2108.07258* (2021). URL: https://arxiv.org/abs/2108.07258.

[3] Tom Brown et al. "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 1877–1901. URL: https://arxiv.org/abs/2005.14165.

[4] Jiawei Chen et al. "Benchmarking Large Language Models in Retrieval-Augmented Generation". In: *arXiv preprint arXiv:2309.01431* (2023). URL: https://arxiv.org/abs/2309.01431.

[5] Tim Dettmers et al. "QLoRA: Efficient Finetuning of Quantized LLMs". In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023. URL: https://arxiv.org/abs/2305.14314.

[6] European Parliament e Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council — General Data Protection Regulation*. 2016. URL: https://eur-lex.europa.eu/eli/reg/2016/679/oj.

[7] Elias Frantar et al. "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers". In: *International Conference on Learning Representations*. 2023. URL: https://arxiv.org/abs/2210.17323.

[8] Sepp Hochreiter e Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.

[9] Edward J. Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models". In: (2022). URL: https://arxiv.org/abs/2106.09685.

[10] Ziwei Ji et al. "Survey of Hallucination in Natural Language Generation". In: *ACM Computing Surveys* 55.12 (2023), pp. 1–38. DOI: 10.1145/3571730.

[11] Albert Q. Jiang et al. "Mistral 7B". In: *arXiv preprint arXiv:2310.06825* (2023). URL: https://arxiv.org/abs/2310.06825.

[12] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474. URL: https://arxiv.org/abs/2005.11401.

[13] Yury A. Malkov e Dmitry A. Yashunin. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2018), pp. 824–836. DOI: 10.1109/TPAMI.2018.2889473.

[14] Niklas Muennighoff et al. "MTEB: Massive Text Embedding Benchmark". In: *arXiv preprint arXiv:2210.07316* (2022). URL: https://arxiv.org/abs/2210.07316.

[15] Zach Nussbaum et al. "Nomic Embed: Training a Reproducible Long Context Text Embedder". In: *arXiv preprint arXiv:2402.01613* (2024). URL: https://arxiv.org/abs/2402.01613.

[16] OpenAI. *GPT-4 Technical Report*. Rapp. tecn. OpenAI, 2023. URL: https://arxiv.org/abs/2303.08774.

[17] Qwen Team e Alibaba Cloud. *Qwen2 Technical Report*. 2024. URL: https://arxiv.org/abs/2407.10671.

[18] Kurt Shuster et al. "Retrieval Augmentation Reduces Hallucination in Conversation". In: *Findings of the Association for Computational Linguistics: EMNLP 2021*. 2021, pp. 3784–3803. URL: https://arxiv.org/abs/2104.07567.

[19] Hugo Touvron et al. "LLaMA: Open and Efficient Foundation Language Models". In: *arXiv preprint arXiv:2302.13971* (2023). URL: https://arxiv.org/abs/2302.13971.

[20] Ashish Vaswani et al. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017. URL: https://arxiv.org/abs/1706.03762.

[21] Jason Wei et al. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models". In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 24824–24837. URL: https://arxiv.org/abs/2201.11903.

Sitografia

[22] Anthropic. *Claude 4 Model Card*. 2025. URL: https://www.anthropic.com/claude/claude-4.

[23] deepset GmbH. *Haystack: Open Source NLP Framework to Build Production-Ready LLM Applications*. 2023. URL: https://haystack.deepset.ai.

[24] Google DeepMind. *Gemini 2.5: Our Most Intelligent Model*. 2025. URL: https://deepmind.google/technologies/gemini/.

[25] Google Research. *TurboQuant: Redefining AI efficiency with extreme compression*. Consultato il: 27-03-2026. Presentato a ICLR 2026. Mar. 2026. URL: https://research.google/blog/turboquant-redefining-ai-efficiency-with-extreme-compression/.

[26] Meta AI. *Llama 4: The Next Generation of Open Foundation Models*. 2025. URL: https://ai.meta.com/blog/llama-4-multimodal-intelligence/.

[27] Mistral AI. *Mistral Large 3*. 2025. URL: https://mistral.ai/news/mistral-large-3/.

[28] Ollama Contributors. *Ollama: Get Up and Running With Large Language Models Locally*. 2023. URL: https://ollama.com.

[29] OpenAI. *Learning to Reason with LLMs*. 2024. URL: https://openai.com/index/learning-to-reason-with-llms/.

[30] Qdrant Contributors. *Qdrant: Vector Database for the Next Generation of AI Applications*. 2023. URL: https://qdrant.tech.

[31] Qwen Team e Alibaba Cloud. *Qwen3 Technical Report*. 2025. URL: https://qwenlm.github.io/blog/qwen3/.